



LLM 初心者向け学習資料 - 完全ガイド

Python 開発経験者向け | 図式を活用した見やすく分かりやすい資料



資料セット完成

このプロジェクトでは、LLM に関する知識ゼロの状態から、実装レベルまで学べる **5段階の総合学習資料** が完成しました。



作成された資料一覧

| # | ファイル | 内容 | 時間 | レベル | |---|-----|-----|-----|-----| | 1 | **01LLMbasicsbeginnersguide.md** | LLM の基本概念、Transformer 解説 | 1.5h | ★ 初級 | | 2 | **02projectoverview_diagram.md** | システムアーキテクチャ、全体像、19フェーズロードマップ | 1.5h | ★ 初級 | | 3 | **03setupandhandson.md** | 環境構築、最初の推論、Tokenizer 実装 | 2h | ★★ 中級 | | 4 | **04inferencepipeline_analysis.md** | 推論パイプライン詳細、ハイパーパラメータ、最適化 | 1.5h | ★★ 中級 | | 5 | **05advancedimplementation_guide.md** | RAG・ファインチューニング、スコアリング実装 | 1.5h | ★★★ 上級 | | 6 | **LLMlearningdashboard.html** | インタラクティブなダッシュボード（ブラウザで開く） | - | 全レベル | | 7 | **LLMHandson_Practice.ipynb** | Jupyter で実装体験（実行可能なコード） | 1.5h | ★★ 中級 |

合計学習時間目安: 9時間〜12時間



クイックスタート

1 初めに確認すべきこと

```
bash
```

プロジェクトディレクトリへ移動

```
cd /home/abemc/project_root
```

Python 環境が準備されているか確認

```
python --version python -c "import torch, transformers; print('✓ OK')"
```

2 学習の推奨順序

第1日目 (2-3時間): |— README を読む (このファイル) |— LLMlearningdashboard.html をブラウザで開く |— 01LLMbasicsbeginnersguide.md で基本概念を学ぶ

第2日目 (2-3時間): |— 02projectoverview_diagram.md でアーキテクチャを理解 |— 03setupandhandson.md で環境構築・最初の推論 |— LLMHandson_Practice.ipynb で実装体験

第3日目 (2-3時間): |— 04inferencepipeline_analysis.md で推論パイプライン理解 |— 05advancedimplementation_guide.md で応用実装 |— プロジェクトコード (autonomousragagent.py) を読む`

3 ダッシュボードの開き方

最も簡単な方法: ` bash

ブラウザで直接開く

open docs/07学習資料/LLMlearning_dashboard.html

または

python -m http.server 8000

ブラウザで http://localhost:8000/docs/07学習資料/LLMlearning_dashboard.html

`

📖 各資料の詳細

段階1: 基本概念 (1階層目)

資料1: LLMbasicsbeginners_guide.md

学習内容: - 従来プログラミング vs AI の違い - LLM とは何か - ニューラルネットワークの基本 - Transformer アーキテクチャ (Attention + Feed-Forward) - 学習フェーズ (事前学習 → ファインチューニング → RLHF)

図式: - 従来プログラミングと機械学習の比較フロー図 - LLM の推論パイプライン (8ステップ) - Attention メカニズムの詳細図解 - 学習フェーズのタイムライン

実装: コードはなし (概念理解重視)

資料2: projectoverviewdiagram.md

学習内容: - システムの5つの主要コンポーネント - 自立型 RAG Agent - RAG (検索拡張生成) - 自律性スコアラー - ファインチューニング - 知識ベース (Corpus) - Phase 1~19 のロードマップ - 技術スタック (PyTorch + Hugging Face + FastAPI など)

図式: - システム全体アーキテクチャ図 - 各コンポーネントの詳細図 - 学習〜推論の完全フロー - 19フェーズのGantt
チャート

実装: なし

段階2: ハンズオン実装（2階層目）

資料3: `setupandhands_on.md`

学習内容: - 環境構築手順 - 最初の推論（感情分析、テキスト生成） - Tokenizer の仕組み - 埋め込みベクトルの理解

コード例: `` python`

例1: 感情分析

`classifier = pipeline("sentiment-analysis") result = classifier("I love this!")`

出力: `[{'label': 'POSITIVE', 'score': 0.9998}]`

例2: Tokenizer

`tokenizer = BertTokenizer.from_pretrained('bert-base-uncased') tokens = tokenizer.tokenize("Hello world")`

出力: `['hello', 'world']`

```

**図式:** - 推論パイプライン図 - Tokenizer の処理フロー - Embedding 層の説明図

**時間:** 約2時間（コード実行含む）

---

**資料4:** `inferencepipelineanalysis.md`

**学習内容:** - 推論の6ステップ詳細解析 - ハイパーパラメータの効果 - Temperature（多様性制御） - Top-K・Top-P（フィルタリング） - Beam Search（検索方法） - KV キャッシング - メモリ最適化テクニック - トラブルシューティング

**コード例:** `` python`

# Temperature の効果比較

```
for temperature in [0.5, 0.7, 1.0, 1.5]: output = model.generate(input_ids, temperature=temperature, do_sample=True)
print(f"Temperature {temperature}: {output}")
```

図式: - 推論パイプライン6ステップ図 - パラメータ効果比較表 - KV キャッシング効果図 - デバッグフロー図

時間: 約1.5時間

## 段階3: 実践応用（3階層目）

資料5: [advancedimplementationguide.md](#)

学習内容: - RAG システムの実装（簡易版とベクトル版） - ファインチューニング実装 - 自律性スコアラーの実装（9つの基準） - フィードバックループの構築 - プロジェクトカスタマイズ方法

コード例: 

```
` python
```

# RAG の実装

```
class SimpleRAG: def retrievedocuments(self, query, topk=2): # ベクトル検索実装 similarities =
cosinesimilarity(queryembedding, docs) return topkdocs
```

# ファインチューニング

```
fine_tuner = FineTuner() finetuner.finetune(traindataset, outputdir="./model")
```

# 自律性スコアリング

```
scorer = AutonomyScorer() overallscore = scorer.calculateoverall_score(metrics)
```

図式: - RAG フロー図 - ファインチューニング効果図 - 自律性の9つの基準図解 - フィードバックループ図

時間: 約1.5時間

## 段階4: インタラクティブダッシュボード

[LLMlearningdashboard.html](#)

機能: - 📊 学習進捗の可視化（プログレスバー） - 📁 タブ別の資料ナビゲーション - 🎯 理解度チェックリスト - 📈 統計表示（完了した段階、資料数、推定時間） - ⚡ クイックリファレンス（コード例）

使い方: 

```
` bash
```

# ブラウザで開く

---

open LLMlearningdashboard.html

## または Web サーバーで提供

---

python -m http.server 8000

**http://localhost:8000/**

**LLMlearningdashboard.html**

---

、

---

### 段階5: Jupyter Notebook 演習

LLMHandson\_Practice.ipynb

含まれるセクション: 1.  セットアップ（必要なライブラリのインストール確認） 2.  最初の推論（Hello LLM）  
3.  Tokenizer 詳細解析 4.  埋め込みベクトルの可視化 5.  簡易 RAG システム実装 6.  自律性スコアリング実装 7.  インタラクティブな演習問題

実行方法: `、 bash jupyter notebook docs/07学習資料/LLMHandsonPractice.ipynb 、`

各セルの実行時間: 約 15〜30分

---



### 学習のコツ

#### 効果的な学習方法

1. **図式をじっくり見る** - 各図の矢印の方向を追う - コンポーネント間の関係を理解 - 自分で図を描き直してみる
2. **コードは手でタイプ** - コピペではなく、実装しながら理解 - エラーが出たら、その原因を考える
3. **段階ごとに理解度チェック** - 各資料の「理解度チェック」セクションを実施 - 説明できなければ、前のセクションに戻る
4. **実装と理論を循環** - 理論を学ぶ → コード例を実行 → 実装 → 理論に戻る

#### よくある失敗

-  コードを読むだけで実行しない -  図式を理解せずに進む -  一度に全部覚えようとする -  難しいセクションをスキップする

## 🌟 推奨方法

- 📝 学んだことをノートに書く - 💬 他の人に説明してみる - 🔄 複数回繰り返す - 🎯 小さな目標を立てる

---

## 🎓 理解度の目安

### 段階1 完了時の目安

- [ ] LLM が「次の単語予測」の繰り返しだと説明できる - [ ] Transformer の構造が図で描ける - [ ] このプロジェクトの全体像が説明できる

### 段階2 完了時の目安

- [ ] Tokenizer の動作が説明できる - [ ] 推論パイプラインの6ステップが列挙できる - [ ] ハイパーパラメータが結果に与える影響が説明できる - [ ] 簡単な推論コードが自分で書ける

### 段階3 完了時の目安

- [ ] RAG システムの「検索」と「生成」の流れが説明できる - [ ] ファインチューニングの目的と方法が説明できる - [ ] 自律性スコアラーの9つの基準が説明できる - [ ] RAG システムをカスタマイズできる

---

## 🚀 次のステップ

### 段階3 完了後は？

1. プロジェクトコードの読解 - `autonomousragagent.py` を読む - 各関数の実装を理解
2. カスタマイズ - 自分のデータセットを追加 - 新しい評価指標を実装 - UI をカスタマイズ
3. 本番環境への展開 - Docker コンテナ化 - API の公開 - スケーリング対応
4. 継続学習 - より高度なモデルを試す - マルチモーダルモデルを学ぶ - LLM のセキュリティを学ぶ

---

## 📞 サポート・フィードバック

### 質問がある場合

1. 該当する資料の「理解度チェック」を再確認
2. コード例を実行して動作を確認
3. Issue を作成して質問

### フィードバック

```
` bash python docs_manager.py --feedback `
```

### バグ報告

- 資料内の誤字・誤植 - コード例のエラー - 図式の不明確さ

---

## 学習資料の統計

| 項目 | 数値 | |-----|-----| | Markdown ファイル数 | 5 | | HTML ダッシュボード | 1 | | Jupyter Notebook | 1 | | 図式 (Mermaid) 数 | 40+ | | コード例の数 | 50+ | | 推定学習時間 | 9〜12 時間 | | 対象レベル | 初級〜上級 |

## 成功指標

このプロジェクトを完了した時点で、以下ができるようになります：

✓ LLM の基本原理を説明できる ✓ Hugging Face Transformers を使って推論ができる ✓ Tokenizer と Embedding の仕組みが理解できる ✓ RAG システムが実装できる ✓ ファインチューニングの流れが理解できる ✓ 自律性スコアリングができる ✓ プロジェクトのコードがカスタマイズできる

## 学習を始めましょう！

推奨開始方法:

```
` bash
```

## ステップ1: ダッシュボードを開く

```
open docs/07学習資料/LLMlearning_dashboard.html
```

## ステップ2: 最初の資料を読む

```
cat docs/07学習資料/01LLMbasicsbeginners_guide.md
```

## ステップ3: コードを実装

```
jupyter notebook docs/07学習資料/LLMHandsonPractice.ipynb`
```

頑張ってください！ 

最終更新: 2026年4月22日 作成者: GitHub Copilot 対象: Python 開発経験者（LLM 知識ゼロ）